Object_Detection

2D

1. Indirect methods (two-stage): Faster R-CNN
2. Direct methods (one-stage):
    1. anchor based: YOLO, RetinaNet
    2. anchor-free/one anchor: , CenterNet, FCOS
3. Faster R-CNN
    1. Faster RCNN uses VGG16 backbone.
    2. ROI pooling is `tf.image.crop_and_resize`
        1. Extracts crops from the input image tensor and resizes them using bilinear sampling or nearest neighbor sampling (possibly with aspect ratio change) to a common output size specified by `crop_size`. This is more general than the `crop_to_bounding_box` op which extracts a fixed size slice from the input image and does not allow resizing or aspect ratio change. The crops occur first and then the resize.
        2. Returns a tensor with `crops` from the input `image` at positions defined at the bounding box locations in `boxes`. The cropped boxes are all resized (with bilinear or nearest neighbor interpolation) to a fixed `size = [crop_height, crop_width]`. The result is a 4-D tensor `[num_boxes, crop_height, crop_width, depth]`. The resizing is corner aligned. In particular, if `boxes = [[0, 0, 1, 1]]`, the method will give identical results to using `tf.compat.v1.image.resize_bilinear()` or `tf.compat.v1.image.resize_nearest_neighbor()`(depends on the `method` argument) with `align_corners=True`.
        3. ROI pooling is just a special case of the SPP layer with one pyramid level
            1. ROI in conv feature map: 21x14->3x2 max pooling with stride(3, 2)->output: 7x7
            2. ROI in conv feature map: 35x42->5x6 max pooling with stride(5, 6)->output: 7x7
            3. Related: see how kernel size and stride are computed dynamically in SPPNet in Convolution
    3. ROI pooling vs ROI align
        1. as part of quantization in ROI pooling it losses a lot of data in the process.
        2. ROI align does not use quantization, but instead it uses bilinear sampling.
        3. after that apply max pooling or avg pooling.
    4. Code: roi_pooling/modules/roi_pool_py.py

```python
import torch
import torch.nn as nn
from torch.autograd import Variable
import numpy as np


class RoIPool(nn.Module):
    def __init__(self, pooled_height, pooled_width, spatial_scale):
        super(RoIPool, self).__init__()
        self.pooled_width = int(pooled_width)
        self.pooled_height = int(pooled_height)
        self.spatial_scale = float(spatial_scale)

    def forward(self, features, rois):
        batch_size, num_channels, data_height, data_width = features.size()
        num_rois = rois.size()[0]
        outputs = Variable(torch.zeros(num_rois, num_channels, self.pooled_height, self.pooled_width)).cuda()

        for roi_ind, roi in enumerate(rois):
            batch_ind = int(roi[0].data[0])
            roi_start_w, roi_start_h, roi_end_w, roi_end_h = np.round(
                roi[1:].data.cpu().numpy() * self.spatial_scale).astype(int)
            roi_width = max(roi_end_w - roi_start_w + 1, 1)
            roi_height = max(roi_end_h - roi_start_h + 1, 1)
            bin_size_w = float(roi_width) / float(self.pooled_width)
            bin_size_h = float(roi_height) / float(self.pooled_height)

            for ph in range(self.pooled_height):
                hstart = int(np.floor(ph * bin_size_h))
                hend = int(np.ceil((ph + 1) * bin_size_h))
                hstart = min(data_height, max(0, hstart + roi_start_h))
                hend = min(data_height, max(0, hend + roi_start_h))
                for pw in range(self.pooled_width):
                    wstart = int(np.floor(pw * bin_size_w))
                    wend = int(np.ceil((pw + 1) * bin_size_w))
                    wstart = min(data_width, max(0, wstart + roi_start_w))
                    wend = min(data_width, max(0, wend + roi_start_w))

                    is_empty = (hend <= hstart) or(wend <= wstart)
```

```
                if is_empty:
                    outputs[roi_ind, :, ph, pw] = 0
                else:
                    data = features[batch_ind]
                    outputs[roi_ind, :, ph, pw] = torch.max(
                        torch.max(data[:, hstart:hend, wstart:wend], 1)[0], 2)[0].view(-1)

        return outputs
```

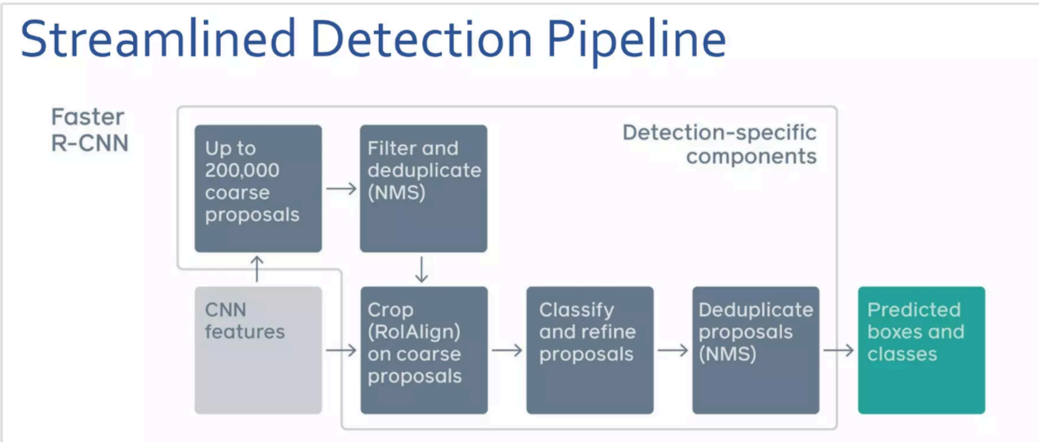
5. Code: roi_pooling/functions/roi_pool.py

```
import torch
from torch.autograd import Function
from .._ext import roi_pooling


class RoIPoolFunction(Function):
    def __init__(self, pooled_height, pooled_width, spatial_scale):
        self.pooled_width = int(pooled_width)
        self.pooled_height = int(pooled_height)
        self.spatial_scale = float(spatial_scale)
        self.output = None
        self.argmax = None
        self.rois = None
        self.feature_size = None

    def forward(self, features, rois):
        batch_size, num_channels, data_height, data_width = features.size()
        num_rois = rois.size()[0]
        output = torch.zeros(num_rois, num_channels, self.pooled_height, self.pooled_width)
        argmax = torch.IntTensor(num_rois, num_channels, self.pooled_height, self.pooled_width).zero_()

        if not features.is_cuda:
            assert False, 'feature not in CUDA'
            _features = features.permute(0, 2, 3, 1)
            roi_pooling.roi_pooling_forward(self.pooled_height, self.pooled_width, self.spatial_scale,
                                            _features, rois, output)
            # output = output.cuda()
        else:
            output = output.cuda()
            argmax = argmax.cuda()
            roi_pooling.roi_pooling_forward_cuda(self.pooled_height, self.pooled_width, self.spatial_scale,
                                                 features, rois, output, argmax)
            self.output = output
            self.argmax = argmax
            self.rois = rois
            self.feature_size = features.size()

        return output

    def backward(self, grad_output):
        # TODO: roi_pooling backward

        assert(self.feature_size is not None and grad_output.is_cuda)

        batch_size, num_channels, data_height, data_width = self.feature_size

        grad_input = torch.zeros(batch_size, num_channels, data_height, data_width).cuda()
        roi_pooling.roi_pooling_backward_cuda(self.pooled_height, self.pooled_width, self.spatial_scale,
                                              grad_output, self.rois, grad_input, self.argmax)

        # print grad_input

        return grad_input, None
```

6. Code: roi_pooling/modules/roi_pool.py

```
from torch.nn.modules.module import Module
from ..functions.roi_pool import RoIPoolFunction


class RoIPool(Module):
    def __init__(self, pooled_height, pooled_width, spatial_scale):
        super(RoIPool, self).__init__()

        self.pooled_width = int(pooled_width)
        self.pooled_height = int(pooled_height)
        self.spatial_scale = float(spatial_scale)

    def forward(self, features, rois):
        return RoIPoolFunction(self.pooled_height, self.pooled_width, self.spatial_scale)(features, rois)
```

7. Faster RCNN(RPN + Fast RCNN)
    1. Insert a Region Proposal Network (RPN) after the last convolutional layer -> using GPU!
    2. RPN trained to produce region proposals directly; no need for external region proposals.
    3. after RPN, use ROI pooling and an upstream classifier and bbox regressor just like Fast RCNN.
8. RPN
    1. slide a small window on the feature map

   2. build a small network for
      1. classifying object or not-object
      2. regressing bbox locations
   3. position of the sliding window provides localization information with reference to the image
   4. box regression provides finer localization information reference to this sliding window
   5. uses k anchor boxes at each location
   6. anchors are translation invariant: use the same ones at every location
   7. regression gives offsets from anchor boxes
   8. classification gives the probability that each (regressed) anchor shows an object.
   9. fully convolutional network
      1. intermediate layer: 256 or 512, 3x3 filter, stride 1, padding 1
      2. cls layer: 18(9x2), 1x1 filter, stride 1, padding 0
      3. reg layer: 36(9x4), 1x1 filter, stride 1, padding 0
   10. anchors as references:
      1. anchors: bbox priors
      2. multi-scale/size anchors
         1. multiple anchors are used at each position: 3 scale (128x128, 256x256, 512x512) and 3 aspect ratios (2:1, 1:1, 1:2) yield 9 anchor combinations
      3. each anchor has its own prediction function
      4. single-scale features, multi-scale predictions
      5. RPN loss function:
         1.
      6. 4-step alternating training
      7. let M0 be an ImageNet pre-trained network
         1. train_rpn(M0)->M1 # Train an RPN initialized from M0, get M1
         2. generate proposals(M1)->P1 # Generate training proposals P1 using RPN M1
         3. train_fast_rcnn(M0, P1)->M2 # Train Fast RCNN M2 on P1 initialized from M0
         4. train_rpn_frozen_conv(M2)->M3 # Train RPN M3 from M2 without changing conv layers
         5. generate_proposals(M3)->P2
         6. train_fast_rcnn_frozen_conv(M3, P2)->M4 # Conv layers are shared with RPN M3
         7. return add_rpn_layers(M4, M3.RPN) # Add M3's RPN layers to Fast RCNN M4
      9. 

Faster R-CNN

4. YOLO
   1. YOLO treats object detection as a **regression problem**. Specifically, we encode the gt bbox centers as offsets from the top-left corner of each cell in the feature map into 0-1 range (as we've known neural networks like to work with values in the range -1 to 1 or 0 to 1). The network will then learn to regress these offsets which is in the range 0-1.
   2. The image is resized to (image_size, image_size) before input to the backbone. For example image_size=224 or 448 are commonly used (224 or 448 for Resnet backbone, that outputs 7x7 or 14x14 feature map).
   3. In yolov1 input size is 448 x 448, **downsampling factor** is 64, output grid size is 7 x 7.
   4. In yolov2, one max-pooling layer is removed. Input size is 448 x 448, downsampling factor is 32, output grid size is 14 x 14.

5. The TLWH box format from COCO is converted to $x_1y_1x_2y_2$ format and divided by (w,h,w,h).
6. The box encoder purpose is to provide (image, target) pair for training. In box encoder the boxes in $x_1y_1x_2y_2$ format is converted to x_cy_cwh format. The offset from the center of each bbox to the top-left corner of each grid cell which is $delta\_xy$ is computed by :
    1. For instance, if S=num_cell=7 (or 14), then cell_size = 1/7.
    2. To convert to grid cell coordinate divide x_cy_c by cell_size. To convert back to image coordinate multiply by $cell\_size$.
    3. In grid cell coordinate, $ij = floor(cxcy\_sample/cell\_size)$, top-left corner (in image coordinate): $xy = ij * cell\_size$. The offset in grid cell coordinate is $delta\_xy = (cxcy\_sample - xy)/cell\_size$. Now, $delta\_xy$ is in the range [0,1] in grid cell coordinate and is to be regressed by the network.
    4. essentially, same as getting the offset in CenterNet: let $cell\_size = c$ $(p_x - \lfloor \frac{p_x}{c} \rfloor * c)/c = \frac{p_x}{c} - \lfloor \frac{p_x}{c} \rfloor$, same with y.
7. YOLO outputs a feature map tensor of size $(S, S, B * 5 + C)$. B is the number of candidate boxes. Note: there are no anchor boxes in YOLOv1.
8. why 5? because the box is of the format $[\delta_x, \delta_y, w, h, conf]$ plus C possible classes for the one box predicted per grid cell.
9. In YOLOv1 it is assumed that there is only one true box per grid cell. Therefore, to match the size of the output tensor (for computing loss). In the third axis of size $B * 5 + C$ only one ground truth box is used and is repeated B consecutive times throughout the third axis.
10. To recover $x_1y_1x_2y_2$ format to compute IOU, because width and height of box is always in image coordinate it does not need to be multiplied by $cell\_size$:
    1. $box\_xyxy[:, :2] = box1[:, :2] * cell\_size - 0.5 * box1[:, 2:4]$
    2. $box\_xyxy[:, 2:4] = box1[:, :2] * cell\_size + 0.5 * box1[:, 2:4]$
    3. This is done so because when combining the two information they should share the same coordinate scale.
    4. Note: normally a function to compute IOU expects $x_1y_1x_2y_2$ format.
11. Box width and height are regressed directly as after normalization by image_size they are in the range[0,1] in image coordinates.
12. Bipartite matching: During training, bounding box assignment is done to assign one candidate ground truth box (since there are B copies of the same ground truth box) to B prediction boxes in each grid cell based on their IOUs. Note: not all B predicted boxes may ever get to be assigned during training. Note: This part is replaced with Hungarian algorithm in DETR, which is used during training only.
13. The last activation function in the model is a Sigmoid function.
14. The loss function is in the form:
15. $L = L_{Localisation} + \lambda_{coobj} * L_{coobj} + \lambda_{noobj} * (L_{noobj} + L_{coo\_no\_response}) + L_{classification}$
16. What I learned from implementing YOLOv1 from scratch:
    1. Training to improve mAP is tough. After 100 epochs the model only achieved ~2% mAP on the CODA dataset.
    2. The idea of using anchor boxes becomes obvious.
    3. Ways to improve mAP include hyperparameter tuning such as grid search using wandb sweeps.
    4. example, box encoder:
        1. box's width and height normalized to the entire image size (this is done after data augmentation, with some operations modifying both the image and bbox sizes i.e. the bbox coordinates, in xyxy format, is normalized by the sizes of the image after data augmentation $bbox * scale\_tensor$).
        2. center coordinates x, y: normalized wrt each grid cell.
        3. image_size=(h, w)=(448, 448)
        4. grid: (7, 7)
        5. grid cell size: (448/7, 448/7)= (64, 64)
        6. centers: (c_x, c_y): (70, 140)
        7. bbox height and width: (b_h, b_w): (150, 120)
        8. normalized box
            1. height: 150/448,
            2. width: 120/448,
        9. x-coordinate: (70-floor(70/64)x64)/64= (70-64)/64
        10. y-coordinate: (140-floor(140/64)x64)/64= (140-128)/64
17. yolov1 box encoder: https://github.com/Liang-ZX/HKU-DASC7606-A1/blob/main/src/data/dataset.py

```
import os
import json
```

```python
import random
import numpy as np

import torch
import torch.utils.data as data

import cv2


CAR_CLASSES = ['Pedestrian', 'Cyclist', 'Car', 'Truck',
               'Tram']

COLORS = {'Pedestrian': (0, 0, 0),
          'Cyclist': (128, 0, 0),
          'Car': (0, 128, 0),
          'Truck': (128, 128, 0),
          'Tram': (0, 0, 128)}


class Dataset(data.Dataset):
    image_size = 448

    def __init__(self, args, split, transform):
        print('DATASET INITIALIZATION')
        self.args = args
        root = args.dataset_root
        self.root_images = os.path.join(root, split, 'image')
        if split == "train":
            self.train = True
        else:
            self.train = False

        self.transform = transform
        self.f_names, self.boxes, self.labels = [], [], []
        self.mean = [123.675, 116.280, 103.530]  # RGB
        self.std = [58.395, 57.120, 57.375]
        annotation_path = os.path.join(root, 'annotations', 'instance_' + split + '.json')
        annotations = load_json(annotation_path)

        for annotation in annotations['annotations']:
            if annotation['image_name'] not in self.f_names:
                if len(self.f_names) != 0:
                    self.boxes.append(torch.Tensor(box))
                    self.labels.append(torch.LongTensor(label))
                box, label = [], []
                self.f_names.append(annotation['image_name'])

            bbox = annotation['bbox']
            x1, y1, x2, y2 = float(bbox[0]), float(bbox[1]), float(bbox[0] + bbox[2]), float(bbox[1] + bbox[3])
            box.append([x1, y1, x2, y2])
            label.append(int(annotation['category_id']))

        self.boxes.append(torch.Tensor(box))
        self.labels.append(torch.LongTensor(label))

        self.num_samples = len(self.boxes)

    def __getitem__(self, idx):
        f_name = self.f_names[idx]
        img = cv2.imread(os.path.join(self.root_images, f_name))
        boxes = self.boxes[idx].clone()
        labels = self.labels[idx].clone()

        if self.train:
            # img = self.random_bright(img)
            img, boxes = random_flip(img, boxes)
            img, boxes = randomScale(img, boxes)
            img = randomBlur(img)
            img = RandomBrightness(img)
            img = RandomHue(img)
            img = RandomSaturation(img)
            img, boxes, labels = randomShift(img, boxes, labels)
            img, boxes, labels = randomCrop(img, boxes, labels)

        h, w, _ = img.shape
        boxes /= torch.Tensor([w, h, w, h]).expand_as(boxes)
        img = BGR2RGB(img)
        img = subMeanDividedStd(img, self.mean, self.std)
        img = cv2.resize(img, (self.image_size, self.image_size))
        target = self.encoder(boxes, labels)  # S*S*(B*5+C)
        for t in self.transform:
            img = t(img)

        return img, target

    def __len__(self):
        return self.num_samples

    def encoder(self, boxes, labels):
        S, B, C = self.args.yolo_S, self.args.yolo_B, self.args.yolo_C
```

```
        grid_num = S
        target = torch.zeros((grid_num, grid_num, B * 5 + C))
        cell_size = 1.0 / grid_num
        wh = boxes[:, 2:] - boxes[:, :2]
        cxcy = (boxes[:, 2:] + boxes[:, :2]) / 2
        for i in range(cxcy.size()[0]):
            cxcy_sample = cxcy[i]
            ij = (cxcy_sample / cell_size).ceil() - 1
            for kk in range(B):
                target[int(ij[1]), int(ij[0]), kk*5 + 4] = 1
            target[int(ij[1]), int(ij[0]), B*5: ] = torch.zeros(C)
            target[int(ij[1]), int(ij[0]), int(labels[i]) + (B-1)*5+4] = 1
            xy = ij * cell_size
            delta_xy = (cxcy_sample - xy) / cell_size

            for kk in range(B):
                target[int(ij[1]), int(ij[0]), kk*5 + 2 : kk*5 + 4] = wh[i]
                target[int(ij[1]), int(ij[0]), kk*5 : kk*5 + 2] = delta_xy
        return target

### rest of the code is not displayed here
```

18. yolov2 box encoder: https://github.com/kuangliu/pytorch-yolov2/blob/master/encoder.py

```
'''Encode target locations and class labels.'''
import math
import torch

from utils import box_iou, box_nms, meshgrid, softmax


class DataEncoder:
    def __init__(self):
        self.anchors = [(1.3221,1.73145),(3.19275,4.00944),(5.05587,8.09892),(9.47112,4.84053),(11.2364,10.0071)]

    def encode(self, boxes, labels, input_size):
        '''Encode target bounding boxes and class labels into YOLOv2 format.

        Args:
          boxes: (tensor) bounding boxes of (xmin,ymin,xmax,ymax) in range [0,1], sized [#obj, 4].
          labels: (tensor) object class labels, sized [#obj,].
          input_size: (int) model input size.

        Returns:
          loc_targets: (tensor) encoded bounding boxes, sized [5,4,fmsize,fmsize].
          cls_targets: (tensor) encoded class labels, sized [5,20,fmsize,fmsize].
          box_targets: (tensor) truth boxes, sized [#obj,4].
        '''
        num_boxes = len(boxes)
        # input_size -> fmsize
        # 320->10, 352->11, 384->12, 416->13, ..., 608->19
        fmsize = (input_size - 320) / 32 + 10
        grid_size = input_size / fmsize

        boxes *= input_size  # scale [0,1] -> [0,input_size]
        bx = (boxes[:,0] + boxes[:,2]) * 0.5 / grid_size  # in [0,fmsize]
        by = (boxes[:,1] + boxes[:,3]) * 0.5 / grid_size  # in [0,fmsize]
        bw = (boxes[:,2] - boxes[:,0]) / grid_size        # in [0,fmsize]
        bh = (boxes[:,3] - boxes[:,1]) / grid_size        # in [0,fmsize]

        tx = bx - bx.floor()
        ty = by - by.floor()

        xy = meshgrid(fmsize, swap_dims=True) + 0.5  # grid center, [fmsize*fmsize,2]
        wh = torch.Tensor(self.anchors)              # [5,2]

        xy = xy.view(fmsize,fmsize,1,2).expand(fmsize,fmsize,5,2)
        wh = wh.view(1,1,5,2).expand(fmsize,fmsize,5,2)
        anchor_boxes = torch.cat([xy-wh/2, xy+wh/2], 3)  # [fmsize,fmsize,5,4]

        ious = box_iou(anchor_boxes.view(-1,4), boxes/grid_size)  # [fmsize*fmsize*5,N]
        ious = ious.view(fmsize,fmsize,5,num_boxes)               # [fmsize,fmsize,5,N] <-- important line,
notice the broadcasting to num_boxes

        loc_targets = torch.zeros(5,4,fmsize,fmsize)  # 5boxes * 4coords
        cls_targets = torch.zeros(5,20,fmsize,fmsize)
        for i in range(num_boxes):
            cx = int(bx[i])
            cy = int(by[i])
            _, max_idx = ious[cy,cx,:,i].max(0)
            j = max_idx[0]
            cls_targets[j,labels[i],cy,cx] = 1

            tw = bw[i] / self.anchors[j][0]
            th = bh[i] / self.anchors[j][1]
            loc_targets[j,:,cy,cx] = torch.Tensor([tx[i], ty[i], tw, th])
        return loc_targets, cls_targets, boxes/grid_size

    def decode(self, outputs, input_size):
        '''Transform predicted loc/conf back to real bbox locations and class labels.
```

```
        Args:
          outputs: (tensor) model outputs, sized [1,125,13,13].
          input_size: (int) model input size.

        Returns:
          boxes: (tensor) bbox locations, sized [#obj, 4].
          labels: (tensor) class labels, sized [#obj,1].
        '''
        fmsize = outputs.size(2)
        outputs = outputs.view(5,25,13,13)

        loc_xy = outputs[:,:2,:,:]   # [5,2,13,13]
        grid_xy = meshgrid(fmsize, swap_dims=True).view(fmsize,fmsize,2).permute(2,0,1)  # [2,13,13]
        box_xy = loc_xy.sigmoid() + grid_xy.expand_as(loc_xy)  # [5,2,13,13]

        loc_wh = outputs[:,2:4,:,:]  # [5,2,13,13]
        anchor_wh = torch.Tensor(self.anchors).view(5,2,1,1).expand_as(loc_wh)  # [5,2,13,13]
        box_wh = anchor_wh * loc_wh.exp()  # [5,2,13,13]

        boxes = torch.cat([box_xy-box_wh/2, box_xy+box_wh/2], 1)  # [5,4,13,13]
        boxes = boxes.permute(0,2,3,1).contiguous().view(-1,4)    # [845,4]

        iou_preds = outputs[:,4,:,:].sigmoid()  # [5,13,13]
        cls_preds = outputs[:,5:,:,:]  # [5,20,13,13]
        cls_preds = cls_preds.permute(0,2,3,1).contiguous().view(-1,20)
        cls_preds = softmax(cls_preds)  # [5*13*13,20]

        score = cls_preds * iou_preds.view(-1).unsqueeze(1).expand_as(cls_preds)  # [5*13*13,20]
        score = score.max(1)[0].view(-1)  # [5*13*13,]
        print(iou_preds.max())
        print(cls_preds.max())
        print(score.max())

        ids = (score>0.5).nonzero().squeeze()
        keep = box_nms(boxes[ids], score[ids])
        return boxes[ids][keep] / fmsize
```

19. yolov2 dataset.py

```
'''Load image/class/box from a annotation file.

The list file is like:

    img.jpg width height xmin ymin xmax ymax label xmin ymin xmax ymax label ...
'''
from __future__ import print_function

import os
import sys
import os.path

import random
import numpy as np

import torch
import torch.utils.data as data
import torchvision.transforms as transforms

from utils import box_iou
from encoder import DataEncoder
from PIL import Image, ImageOps


class ListDataset(data.Dataset):
    input_sizes = [320 + 32*i for i in range(10)]

    def __init__(self, root, list_file, train, transform):
        '''
        Args:
          root: (str) ditectory to images.
          list_file: (str) path to index file.
          train: (boolean) train or test.
          transform: ([transforms]) image transforms.
        '''
        self.root = root
        self.train = train
        self.transform = transform

        self.fnames = []
        self.boxes = []
        self.labels = []

        self.data_encoder = DataEncoder()

        with open(list_file) as f:
            lines = f.readlines()
            self.num_samples = len(lines)

        for line in lines:
            splited = line.strip().split()
            self.fnames.append(splited[0])
```

```python
            num_boxes = (len(splited) - 3) // 5
            box = []
            label = []
            for i in range(num_boxes):
                xmin = splited[3+5*i]
                ymin = splited[4+5*i]
                xmax = splited[5+5*i]
                ymax = splited[6+5*i]
                c = splited[7+5*i]
                box.append([float(xmin),float(ymin),float(xmax),float(ymax)])
                label.append(int(c))
            self.boxes.append(torch.Tensor(box))
            self.labels.append(torch.LongTensor(label))

    def __getitem__(self, idx):
        '''Load a image, and encode its bbox locations and class labels.

        Args:
          idx: (int) image index.

        Returns:
          img: (tensor) image tensor.
          loc_targets: (tensor) location targets.
          cls_targets: (tensor) class label targets.
          box_targets: (tensor) truth box targets.
        '''
        # Load image and bbox locations.
        fname = self.fnames[idx]
        img = Image.open(os.path.join(self.root, fname))
        boxes = self.boxes[idx].clone()
        labels = self.labels[idx]

        # Data augmentation while training.
        if self.train:
            img, boxes = self.random_flip(img, boxes)
            img, boxes, labels = self.random_crop(img, boxes, labels)

        # Scale bbox locaitons to [0,1].
        w,h = img.size
        boxes /= torch.Tensor([w,h,w,h]).expand_as(boxes)

        input_size = 416
        # input_size = random.choice(self.input_sizes)
        img = img.resize((input_size,input_size))
        img = self.transform(img)

        # Encode data.
        loc_targets, cls_targets, box_targets = self.data_encoder.encode(boxes, labels, input_size)
        return img, loc_targets, cls_targets, box_targets

### rest of the code is not displayed here
```

5. CenterNet
    1. Objects as points.
    2. Starting with image $I \in \mathbb{R}^{W\times H\times 3}$
    3. Training for keypoint classification:
        1. The network will produce a keypoint **heatma**p $\hat{Y} \in [0,1]^{\frac{W}{R}\times\frac{H}{R}\times C}$
            1. R is the output stride, imagine the total effect of all the strides in the convolution operations until the last feature map as output.
            2. C is the no. of keypoint types, for example:
                1. C=17 in human pose estimation
                2. C=80 object categories in object detection (COCO)
            3. use R=4, default in literature. R is **downsampling stride**. Image is **downsampled** R times in resolution.
        2. To construct the target $Y \in [0,1]^{\frac{W}{R}\times\frac{H}{R}\times C}$,
            1. Let the ground truth keypoint in image pixel coordinate be $p = (p_x, p_y)$. Then compute $\tilde{p}$ the lower resolution equivalent of p as:
                1. For example image is of size 28x28x3 divided by 4x4 tiles to become 7x7 grid. Say p=(26,27), R=4,
                    1. $\tilde{p} = \frac{P}{R} = (\frac{26}{4}, \frac{27}{4}) = (6.5, 6.75)$, $\lfloor \frac{P}{R} \rfloor = (6,6)$,
                    2. $\tilde{p} = \frac{P}{R} = (\frac{25}{4}, \frac{24}{4}) = (6.25, 6.)$, $\lfloor \frac{P}{R} \rfloor = (6,6)$.
            2. (x, y) being the coordinate of Y.
            3. Imagine point $(\tilde{p}_x, \tilde{p}_y)$ in the center and 8 points (x_i, y_i) surrounding it.
            4. Then for each class c, splat the cth dimension of Y with:
            5. $Y_{xyc} = \exp\left(-\frac{(x-\tilde{p}_x)^2+(y-\tilde{p}_y)^2}{2\sigma_p}\right)$.
            6. If two gaussians of the *same* class overlap, we take the element wise maximum.
            7. The training objective is pixel wise logistic regression with focal loss:

8. $$L_k = \sum_{xyc} -\frac{1}{N} \begin{cases} (1-\hat{Y}_{xyc})^{\alpha} \log(\hat{Y}_{xyc}) & \text{if } Y_{xyc} = 1 \\ (1-Y_{xyc})^{\beta}(\hat{Y}_{xyc})^{\alpha} \\ \log(1-\hat{Y}_{xyc}) & \text{, otherwise} \end{cases}$$

4. Training for keypoint localisation:
   1. The heatmap predicts both the class (there is one heatmap $\frac{H}{R} \times \frac{W}{R}$ predicted for each class c) and the centers of each bounding box is the 'center' of each Gaussian splatted on the heatmap. However, the centers of the heatmap grid are integers. Therefore, there needs to be another head to predict the offsets so that final bbox predictions have decimal values or as floats.
   2. $$L_{off} = \text{localisation error in x coordinate} + \text{localisation error in y coordinate}$$
   3. $$L_{off} = \frac{1}{N}(\sum_{\tilde{p}_x} |\hat{O}_{\tilde{p}_x} - (\frac{p_x}{R} - \lfloor \frac{p_x}{R} \rfloor)| + \sum_{\tilde{p}_y} |\hat{O}_{\tilde{p}_y} - (\frac{p_y}{R} - \lfloor \frac{p_y}{R} \rfloor)|$$
5. Training to recover width and height:
   1. kth Keypoint: $p_k = (p_x, p_y)$ with bbox:(x_1, y_1, x_2, y_2), $p_k = (\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$
   2. $\hat{S}_k$: predicted bbox width &height, $(\hat{x}_2^{(k)} - \hat{x}_1^{(k)}, \hat{y}_2^{(k)} - \hat{y}_1^{(k)})$
   3. S_k : ground truth width & height, $(x_2 - x_1, y_2 - y_1)$
   4. Without normalizing the scale and directly use raw pixel coordinates.
   5. $$L_{size} = \frac{1}{N} \sum_{p_k} |\hat{S}_{p_k} - S_k|$$
6. Then the overall training loss:
   1. $$L = L_k + \lambda_{size} L_{size} + \lambda_{off} L_{off}$$
   2. we set $lambda_{size} = 0.1$ and $\lambda_{off} = 1$, means penalize object center offset more than fitting object width & height.
7. The final output size is $(\frac{W}{R}, \frac{H}{R}, C+4)$, 4=2+2, 2 values for bbox width and height, and 2 values for the centers p_x and p_y offsets.

6. DETR
   1. Compared to YOLO approach:
      1. Object query in place of anchor boxes.
      2. During training Hungarian matching in place of 'NMS' like bounding box assignment.
      3. As a result DETR is NMS free (no need of NMS during training or inference).
   2. DETR predicts in parallel the final set of detections.
   3. The CNN is used for feature learning, the transformer is used to make predictions.
   4. DETR casts the object detection task as **an image-to-set prediction**.
   5. Given an image, the model must predict **an unordered set of all the objects present**, each represented by its class, along with a tight bounding box surrounding each one.
   6. This formulation is particularly suitable for transformers.
   7. Using transformers and parallel decoding.
   8. Eliminates duplicate predictions.
   9. multiclass: dog: 0.9 -- uses Softmax activation function or cross entropy loss.
   10. multilabel: dog: 0.9, trees: 0.75, -- uses Sigmoid or binary cross entropy loss, set prediction.
   11. Code: dummy example of multi label classification in pytorch.

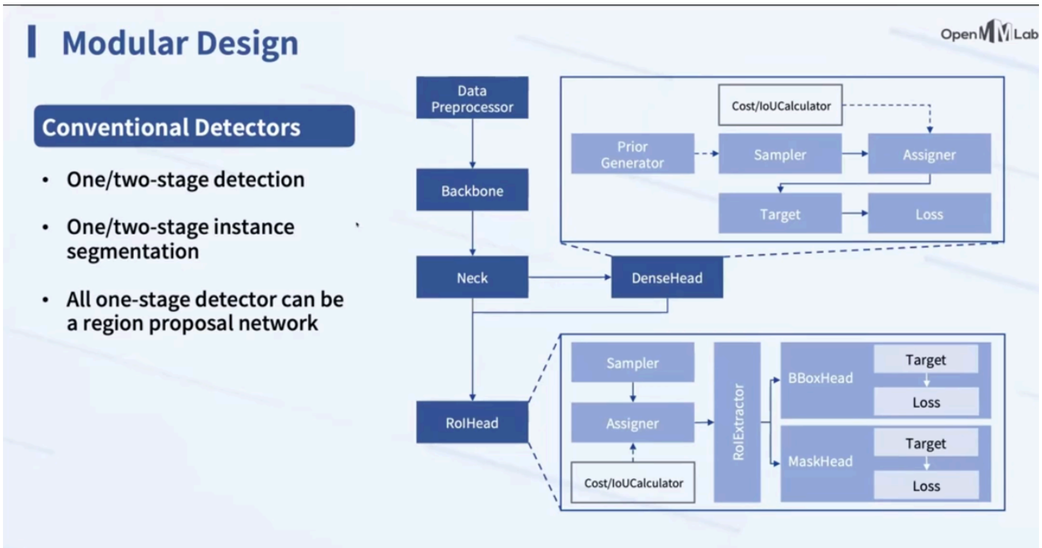
```
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Linear(20, 5) # predict logits for 5 classes
x = torch.randn(1, 20)  # batch_size = 1
y = torch.tensor([[1., 0., 1., 0., 0.]]) # get classA and classC as active

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.SGD(model.parameters(), lr=1e-1)

for epoch in range(20):
    optimizer.zero_grad()
    output = model(x)
    print('output.shape', output.shape)  # (batch_size, n_classes): (1, 5)
    loss = criterion(output, y)
```

```
        loss.backward()
        optimizer.step()
        print('Loss: {:.3f}'.format(loss.item()))
```

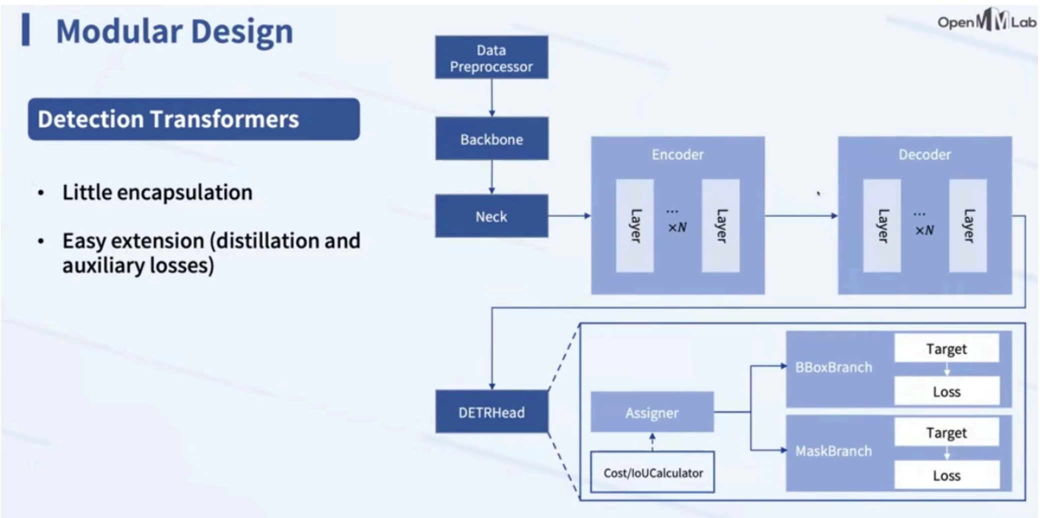
12. Issue with multilabel classification is when the model predicts the same amount of probability for two different objects. In object detection, if the region proposals are too near to each other.
13. In the context of object detection, DETR predicts N number of object boxes and N is much larger than the amount of ground truth bounding boxes G.
14. Bipartite matching is used to match the set of detections N to set of ground truth boxes in G. Note: Bipartite matching is the process of pairing vertices from two sets in such a way that each vertex is paired with at most one vertex from the other set, and the total number of paired vertices is maximized.
15. DETR infers a fixed-size set of N predictions, where N is set to be significantly large than the typical number of objects in an image.
16. Let use denote by y the ground truth set of objects, and $\hat{y} = \{\hat{y}\}_{i=1}^{N}$ the set of N predictions.
17. y also as a set of size N padded with $\emptyset$(no object).
18. To find a bipartite matching between these to sets we search for a permutation of N elements $\sigma \in \mathit{\Gamma}_N$ with the lowest cost:
    1. $\hat{\sigma} = \arg\min_{\sigma \in \Gamma_N} \sum_i^N L_{match}(y_i, \hat{y}_{\sigma(i)})$

7. SPP
    1. Used to adaptively produce feature maps of constant sizes, by recomputing the necessary kernel sizes and strides.
8. FPN
    1. Used to perform multi-scale object prediction.

    mmdetection2d :One-stage detectors can be viewed as a Region Proposal Network in a two-stage network, which is the upper box in this image.



DETR



3D

References:

1. Multi-label classification
2. MSDN, github
3. https://erdem.pl/2020/02/understanding-region-of-interest-ro-i-pooling
4. https://erdem.pl/2020/02/understanding-region-of-interest-part-2-ro-i-align
5. Yolov1, https://www.slideshare.net/slideshow/yolo-v1-241861792/241861792#17
6. Faster R-CNN, https://www.slideshare.net/slideshow/pr12-faster-rcnn170528/78497784 by Jinwon Lee
7. DETR, https://www.slideshare.net/slideshow/pr284-endtoend-object-detection-with-transformersdetr/239265010 by Jinwon Lee